Chapter 9 worked solutions

DS09-01 • Leftist rank and meld

H_1's children have rank 1, so root 3 has rank 2. H_2's child ranks are 1 and 0, so root 4 has rank 1. Meld keeps 3, recursively melds 9 with 4, keeps 4, and attaches 9 as its right child. Children 8,9 both have rank 1, giving 4.rank=2. At 3, left child 7 has rank 1 and right child 4 rank 2. Swap them, giving left 4, right 7, and 3.rank=2. The recurrence N(0)=0 and $N(r) \geq 1 + 2N(r-1)$ yields $2^r - 1$. An increasing-key pure left chain has right rank 0 and left rank at least 0 everywhere. Every nonempty node has rank 1, but height is n−1.

DS09-02 • Binomial carries and amortized insertion

Size 5 has degrees {0,2}; size 3 has {0,1}. Link B_0+B_0 into B_1, combine with the original B_1 into B_2, then with the original B_2 into B_3. Eight nodes remain in one tree. A dedicated insertion adds a root and makes k links, with actual cost at most a(k+1). Root-count change is Δt=1−k. With Φ=a • t, amortized cost is at most a(k+1)+a(1−k)=2a. A complete O(log n) scan even when no carry occurs is not paid by this potential change. The proof requires stopping when the next degree is absent, or a separate argument bounding total scanning.

DS09-03 • The role of Fibonacci cascading cuts

The c cuts increase t by c. The c−1 previously marked ancestors become unmarked roots, while the stopping ancestor may gain one mark. Thus $\Delta\Phi \leq c - 2(c-1) + 2 = 4 - c$. Scaling the potential by a sufficient constant pays constant work per cut and leaves O(1) amortized cost. Without cascading cuts, a child remaining under its parent can lose arbitrarily many children. Its current degree need not be at least its attachment degree minus one. The ith retained child's degree lower bound i−2 fails, so the logarithmic maximum-degree argument no longer follows.

DS09-04 • Pairing operations and one shared potential

The first pass makes 3(child 8), 5(child 9), and 7. The second links 5 with 7, yielding 5(children 7,9), then links 3 with 5, yielding 3(children 5,8). Node 5 retains 7,9. Four links occur.

Chapter 9 worked solutions (continued)

LCRS size s(v) includes right siblings and their descendants, not merely the original multiway subtree. $\Phi = \sum_v \log_2 s(v) \geq 0$ and empty Φ=0. Sum separate potentials for independent heaps. Within delete-min, analyze the ordered forest's virtual sibling chain even when roots are detached into an array; clearing each physical next pointer does not define a different heap potential.

For adjacent components A,B with original sizes a,b≥1 and right suffix R≥0, their initial binary sizes are a+b+R,b+R. Linking leaves the winner with a+b+R and the loser with a+b−1; other sizes are unchanged. Either winner gives $\delta = \log_2(a+b-1) - \log_2(b+R)$. For R>0, put A=a+b,T=A+R. Then $1+\delta = \log_2 \frac{2(A-1)}{b+R} \leq \log_2 \frac{2A}{R} \leq 2\log_2 \frac{T}{R}$, since $2AR \leq (A+R)^2$.

Let N=n−1 and q=⌊k/2⌋. Consecutive pairs satisfy $R_j = T_{j+1}$. If k=2q+1, the last unpaired tree has size r≥1; every R>0, and $q + \Delta\Phi_1 \leq 2\log_2(N/r) \leq 2\log_2 N$. If k=2q, the last pair has R=0 and total size $A_q \geq 2$. The earlier pairs give $(q-1) + \Delta\Phi_{\text{previous}} \leq 2\log_2(N/A_q)$; the last pair has $\delta \leq \log_2 A_q$. Hence $\Delta\Phi_1 \leq -q + 1 + 2\log_2 N$. For q=1 the earlier sum is empty. Do not divide by the last pair's zero suffix.

In the second pass, the accumulated right tree has size b and the next left tree size a. With R=0, $\delta \leq \log_2 \frac{a+b}{b}$. Each new accumulator is the next denominator, so $\Delta\Phi_2 \leq \log_2(N/b_0) \leq \log_2 N$, where b_0 is the rightmost component size. A single component gives an empty sum. Root removal subtracts $\log_2 n$. Thus for k≥2, $\Delta\Phi \leq -q + 3\log_2 n + 1$. Bound actual work by ck+d and use k≤2q+1. One scaled potential $\Psi = C\Phi,\ C \geq 2c$, gives amortized $\text{cost} \leq (2c-C)q + 3C\log_2 n + c + d + C = O(1+\log n)$. The term ck+d includes scanning and all k−1 links. For k=0 deletion removes the singleton, ΔΦ=0. For k=1 there are no links and $\Delta\Phi = -\log_2 n$. Both cost O(1) actual. Reject empty deletion; temporary arrays use O(k) space.

The five-singleton trace has first-pass potential ratios 1/4,1/2 and second-pass ratios 2,4/3. Root removal contributes 1/6, giving total $\Delta\Phi = \log_2(1/18)$. This checks the example; it does not replace the general argument.

Public meld starts with independent heaps. For nonempty sizes a,b and n=a+b, $\Delta\Phi = \log_2 \frac{n(n-1)}{ab} \leq \log_2 n$ because ab≥n−1. An empty operand gives 0. Insert melds a singleton, increasing potential by exactly $\log_2 n$ when the old heap is nonempty. Both operations take O(1) actual work but receive O(1+log(n+1)) amortized charges under this Ψ.

Chapter 9 worked solutions (continued)

Decreasing a root key leaves shape and potential unchanged. Cutting a nonroot x into a separate heap removes its right-sibling suffix from s(x). Its former binary ancestors only lose reachable nodes; other sizes are unchanged. The cut therefore has $\Delta\Phi \leq 0$. Melding back adds at most $\log_2 n$, yielding $O(1+\log(n+1))$ amortized decrease under the same Ψ. Constant actual cut/link work requires a valid handle and maintained parent/prev/next backlinks. A next-only representation may scan for the predecessor.

For initially empty heaps, $\Psi_0=0$ and $\Psi_{\text{final}} \geq 0$, so total actual cost is at most total amortized charges. If N is the maximum live-node count and I,M,D,K,F count insert,meld,delete-min,decrease,find-min, the bound is $O((I+M+D+K)(1+\log_2(N+1))+F)$, plus O(1) per empty-heap creation. This proof does not give constant amortized insert/meld, so it cannot be rewritten as $O(I+M+(D+K)\log N+F)$.

DS09-05 • Workload selection

Build an indexed binary heap of V elements in O(V). With handles, each decrease-key and delete-min takes O(log V), giving O((V+E)log V) plus construction. A Fibonacci heap starting empty performs V inserts, V delete-mins, and E decrease-keys in total amortized O(V log V+E). These abstract costs assume constant-time key operations and omit cache misses, allocation effects, and implementation constants. Pointer-rich Fibonacci structures may cost more per abstract step. Actual comparison requires the same workload and data, not an invented speed ratio from asymptotic formulas.

DS09-06 • Direct erase, handle lifetime, and one shared potential

Initially n=32, with roots v26(key−11,d0),v18(key−10,d0),v1(key1,d5), marked nodes {v17,v25}, and Φ/A=7. Target v29 has parent v25 and children v31(d1, child v32),v30(d0). Cut(v29,v25) lowers v25.degree from 2 to 1. Marked v25 is then cut from v17 and unmarked, lowering v17.degree from 3 to 2. Marked v17 is cut from v1 and unmarked, lowering v1.degree from 5 to 4. Stop at root v1. Now t=6,m=0 and Φ/A=6.

Promote v31,v30 by removing them from v29's ring, clearing parent/mark, and adding them to roots. Target degree becomes zero and t=8. Remove v29, invalidate h29, and decrement n to 31, leaving seven roots. Process snapshot v30d0,v31d1,v17d2,v25d1,v26d0,v18d0,v1d4. Root v25 wins v31 and reaches d2; v17 wins v25 and reaches d3; v26(key−11) wins v30 and reaches d1. Final table: A0=v18,A1=v26,A3=v17,A4=v1. Thus t=4,m=0,Φ/A=4, and min=v26 with key−11.

Chapter 9 worked solutions (continued)

No item or node identity is exchanged. Handles h31,h30 still name their original nodes after promotion and relinking, as do all other undeleted handles. Reusing h29 fails. Equal keys can have distinct payloads and references, so erasing an arbitrary equal key does not implement erase of the specified handle. Reject foreign ownership before node access. Omitting cascade would let already marked nonroot v25 lose another child while remaining attached, violating the degree proof's one-loss premise.

For k cuts, $\Delta(t + 2m) \leq 4 - k$ pays ak+b actual work with one sufficiently large A, giving O(1) amortized cutting. Extraction starts with t′ roots, costs O(t′+d+D+1), ends with at most D+1 roots, and never increases marks. Potential decrease cancels t′, leaving O(d+D+1)=O(1+log(n+1)). Both phases use the same potential so their differences telescope. This trace has ΔΦ/A=−3, but the general bound does not depend on that value. A root target skips cutting; the final node leaves n=0,roots=min=null. Reject empty delete-min and stale handles first. Individual operations may still take linear time.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Chapter 9 worked solutions (continued)

Verification code appendix: leftist.cpp

```
#include <algorithm>
#include <iostream>
#include <memory>
#include <random>
#include <stdexcept>
#include <utility>
#include <vector>

struct Node {
        explicit Node(int item) : key(item) {}
        int key;
        int rank = 1;
        std::unique_ptr<Node> left;
        std::unique_ptr<Node> right;
};

using Heap = std::unique_ptr<Node>;

int rank(const Heap& heap) {
        return heap ? heap->rank : 0;
}

Heap meld(Heap first, Heap second) {
        if (!first) {
                return second;
        }
        if (!second) {
                return first;
        }
        if (first->key > second->key) {
                std::swap(first, second);
        }
        first->right = meld(std::move(first->right), std::move(second));
        if (rank(first->left) < rank(first->right)) {
                std::swap(first->left, first->right);
        }
        first->rank = rank(first->right) + 1;
        return first;
}

void insert(Heap& heap, int key) {
        heap = meld(std::move(heap), std::make_unique<Node>(key));
}

int remove_min(Heap& heap) {
        if (!heap) {
                throw std::out_of_range("empty priority queue");
        }
        const int result = heap->key;
        auto old_root = std::move(heap);
        heap = meld(std::move(old_root->left), std::move(old_root->right));
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Chapter 9 worked solutions (continued)

```
        return result;
}

std::size_t validate(const Heap& heap) {
        if (!heap) {
                return 0;
        }
        if (rank(heap->left) < rank(heap->right) || heap->rank != rank(heap->right) + 1 ||
                (heap->left && heap->left->key < heap->key) ||
                (heap->right && heap->right->key < heap->key)) {
                throw std::runtime_error("leftist invariant was violated");
        }
        return 1 + validate(heap->left) + validate(heap->right);
}

int main() {
        Heap first;
        Heap second;
        std::vector<int> expected;
        std::mt19937 random(20261024);
        for (int i = 0; i < 240; ++i) {
                const int value = static_cast<int>(random() % 101) - 50;
                insert(i % 2 == 0 ? first : second, value);
                expected.push_back(value);
                validate(first);
                validate(second);
        }
        auto combined = meld(std::move(first), std::move(second));
        if (first || second || validate(combined) != expected.size()) {
                throw std::runtime_error("destructive meld ownership or size differs");
        }
        std::sort(expected.begin(), expected.end());
        for (const int value : expected) {
                if (remove_min(combined) != value) {
                        throw std::runtime_error("priority queue output order differs");
                }
                validate(combined);
        }
        bool rejected = false;
        try {
                remove_min(combined);
        } catch (const std::out_of_range&) {
                rejected = true;
        }
        if (!rejected) {
                throw std::runtime_error("empty removal was accepted");
        }
        std::cout << "PASS leftist: 240 inserts, meld ownership, invariant checks, sorted
drain, empty\n";
}
```

Chapter 9 worked solutions (continued)

References and attribution

Ellis Horowitz, Sartaj Sahni, Dinesh Mehta. Fundamentals of Data Structures in C++, 2nd Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study